FlashNorm: fast normalization for LLMs

Nils Graef[*], **Filip Makraduli**, **Andrew Wasielewski**, **Matthew Clapp**
OpenMachine

Abstract

This paper presents FlashNorm, which is an exact but faster implementation of RMSNorm followed by linear layers. RMSNorm [1] is used by many LLMs such as Llama, Gemma, Mistral, and OLMo 2 [2, 3, 4, 5]. FlashNorm also speeds up Layer Normalization [6] and its recently proposed replacement Dynamic Tanh (DyT) [7]. FlashNorm reduces the number of parameter tensors by simply merging the normalization weights with the weights of the next linear layer. Watch our explainer video [8] and see [9, 10, 11, 12] for code and more transformer tricks.

1 Flash normalization

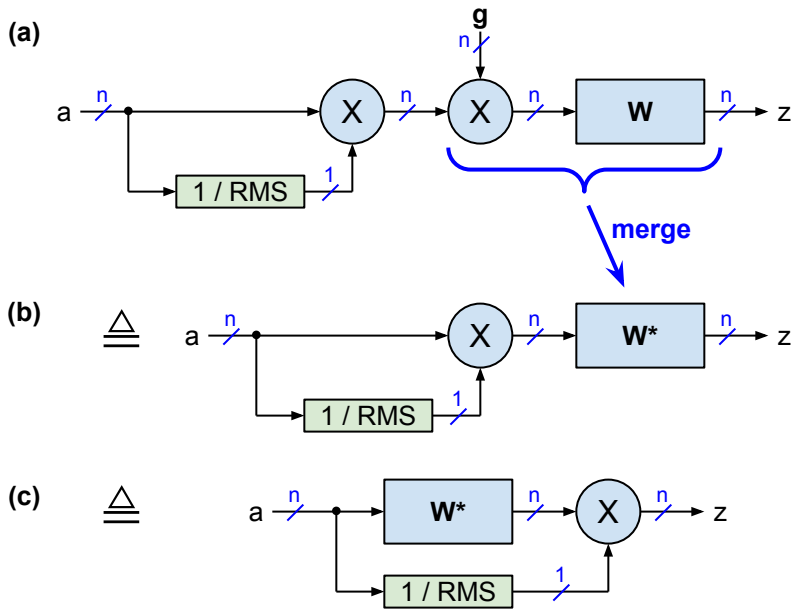

Figure 1: Mathematically identical implementations of RMSNorm followed by a linear layer: (a) unoptimized version with weight matrix $\mathbf{W}$; (b) optimized version with normalization weights g_i merged into the linear layer with new weights $\mathbf{W}^*$; (c) optimized version with deferred normalization. The $\triangleq$ symbol denotes mathematical identity.

RMSNorm [1] normalizes the elements a_i of vector $\vec{a}$ as $y_i = \frac{a_i}{\text{RMS}(\vec{a})} \cdot g_i$ with $\text{RMS}(\vec{a}) = \sqrt{\frac{1}{n}\sum_{i=1}^{n} a_i^2}$ and normalization weights g_i. In transformer [13] and other neural networks, RMSNorm is often followed by a linear layer as illustrated in Fig. 1(a), which we optimize as follows:

[*] info@openmachine.ai

- **Weightless normalization (aka non-parametric normalization)**: We merge the normalization weights g_i into the linear layer with weights $\mathbf{W}$, resulting in a modified weight matrix $\mathbf{W}^*$ with $W^*_{i,j} = g_i \cdot W_{i,j}$ as illustrated in Fig. 1(b). This works for linear layers with and without bias.
- **Deferred normalization**: Instead of normalizing before the linear layer, we normalize after the linear layer, as shown in Fig. 1(c). This only works if the linear layer is bias-free, which is the case for many LLMs such as Llama, Gemma, Mistral, OLMo and OpenELM. Specifically, the output of the linear layer in Fig. 1(b) is $\vec{z} = \left(\vec{a} \cdot \frac{1}{\text{RMS}(\vec{a})}\right) \mathbf{W}^*$, which is identical to $\vec{z} = (\vec{a}\,\mathbf{W}^*) \cdot \frac{1}{\text{RMS}(\vec{a})}$ because matrix multiplication by a scalar is commutative. If the linear layer has a bias at its output, then the normalization (i.e. scaling by $\frac{1}{\text{RMS}(\vec{a})}$) must be done before adding the bias.

In summary, FlashNorm eliminates the normalization weights and defers the normalization to the output of the linear layer, which removes a compute bottleneck described at the end of this paper. Deferring the normalization is similar to Flash Attention [14], where the normalization by the softmax denominator is done after the multiplication of softmax arguments with value projections (V) (so that keys and values can be processed in *parallel*). Therefore, we call our implementation *flash* normalization (or FlashNorm), which allows us to compute the linear layer and RMS$(\vec{a})$ in *parallel* (instead of sequentially).

Mehta et al. report significant changes in the overall tokens-per-second throughput when they modify the layer normalization implementation, which they attribute to a lack of kernel fusion for the underlying GPU. The simplifications presented here reduce the number of operations and thus the number of the individual kernel launches mentioned in [15].

1.1 Support for normalization bias and DyT bias

Layer normalization (LayerNorm) [6] and DyT [7] can have a bias vector $\vec{\beta}$ right after scaling by weights g_i. Figure 2 illustrates how the bias vector $\vec{\beta}$ can be moved to the output of the linear layer and then be added to the bias vector $\vec{c}$ of the linear layer, resulting in the new bias term $\vec{c}^* = \vec{c} + \vec{\beta}\,\mathbf{W}$, see Fig. 2(b). After this elimination of $\vec{\beta}$, the normalization weights g_i can be merged into the linear layer as described in the previous section and illustrated in Fig. 1(b).

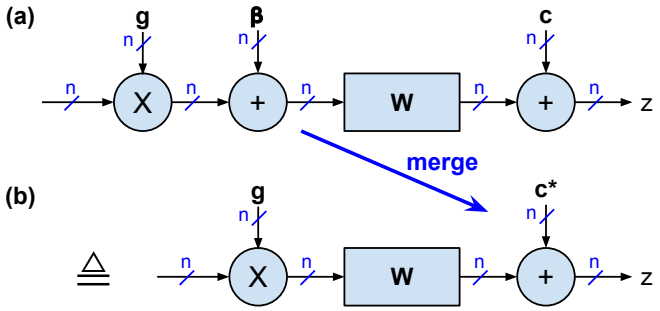

Figure 2: Elimination of bias vector $\vec{\beta}$: (a) Before elimination with $\vec{\beta}$ between normalization weights $\vec{g}$ and linear layer. (b) Optimized version with new bias term $\vec{c}^* = \vec{c} + \vec{\beta}\,\mathbf{W}$ at the output.

1.2 Merging mean centering into a preceding linear layer

Note that LayerNorm consists of mean centering followed by RMSNorm. If the mean centering is preceded by a linear layer with weight matrix $\mathbf{V}$, then we can eliminate the entire mean centering by modifying the weight matrix as explained in this section. Fig. 3(a) shows the weight matrix $\mathbf{V}$ followed by the mean centering, which is followed by RMSNorm.

The mean μ is calculated from the linear layer outputs y_j as $\mu = \frac{1}{n}\sum_{j=1}^{n} y_j$. Note that $\vec{y} = \vec{x}\,\mathbf{V}$, i.e. $y_j = \sum_{i=1}^{n} x_i v_{i,j}$ where $v_{i,j}$ are the weights of matrix $\mathbf{V}$. Plugging the last equation into the μ

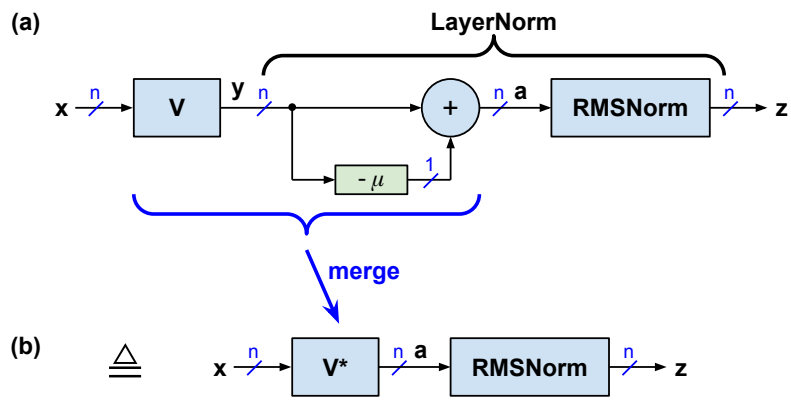

Figure 3: Elimination of mean centering: (a) Original weight matrix $\mathbf{V}$ followed by mean centering. (b) Optimized version where the mean centering is merged into the modified weight matrix $\mathbf{V}^*$.

expression lets us calculate μ directly from the input $\vec{x}$ as

$$\mu = \frac{1}{n}\sum_{j=1}^{n}\sum_{i=1}^{n} x_i v_{i,j} = \frac{1}{n}\sum_{i=1}^{n} x_i \left[\sum_{j=1}^{n} v_{i,j}\right] = \frac{1}{n}\sum_{i=1}^{n} x_i s_i$$

where we define vector $\vec{s}$ with $s_i = \sum_{j=1}^{n} v_{i,j}$ the sum of row i of weight matrix $\mathbf{V}$. In other words, μ is the inner-product of vectors $\vec{x}$ and $\vec{s}$ divided by n. The outputs a_j of the mean centering are

$$\vec{a}_j = y_j - \mu = \sum_{i=1}^{n} x_i v_{i,j} - \mu = \sum_{i=1}^{n} x_i v_{i,j} - \frac{1}{n}\sum_{i=1}^{n} x_i s_i = \sum_{i=1}^{n} x_i \left(v_{i,j} - \frac{1}{n} s_i\right) = \sum_{i=1}^{n} x_i v^*_{i,j}$$

From the last identity follows that the new weights $v^*_{i,j}$ of matrix $\mathbf{V}^*$ of Fig. 3(b) are computed as $v^*_{i,j} = v_{i,j} - \frac{1}{n} s_i$. This trick can be used to retrofit existing LayerNorm models with RMSNorm without any retraining.

2 Flash normalization for FFN

For the feed-forward networks (FFN) of LLMs, the linear layers at the FFN input usually have more output channels than input channels. In this case, deferring the normalization requires more scaling operations (i.e. more multiplications). This section details ways to reduce the number of scaling operations for bias-free FFNs.

2.1 Flash normalization for FFNs with ReLU

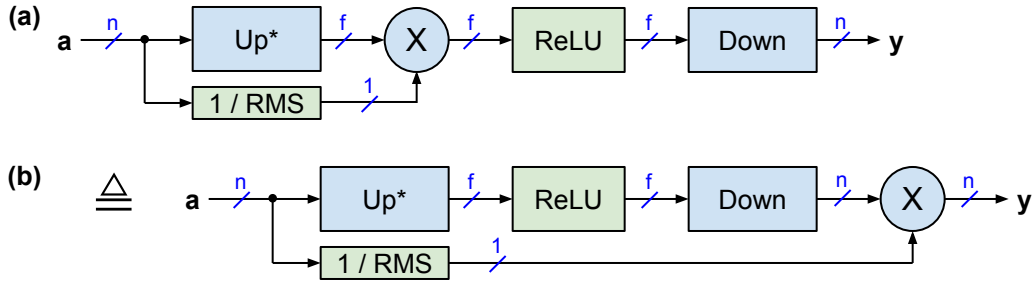

Figure 4: FFN with ReLU and preceding flash normalization: (a) unoptimized version; (b) optimized version where the normalization is deferred to the output of the FFN. Up and Down denote the linear layers for up and down projections.

Even though ReLU is a nonlinear function, multiplying its argument by a non-negative scaling factor s is the same as scaling its output by s, i.e. $\mathrm{ReLU}(s \cdot \vec{a}) = s \cdot \mathrm{ReLU}(\vec{a})$ for $s \geq 0$ [16]. Because of

this scale-invariance, we can defer the normalization to the output of the FFN as illustrated in Fig. 4(b), which saves $f - n$ multipliers.

2.2 Flash normalization for FFNs with GLU variant

Fig. 5(a) shows an FFN with a GLU variant [17] and flash normalization at its input. The flash normalization requires two sets of f multipliers at the outputs of the Gate and Up linear layers in Fig. 5(a). One set can be deferred to the FFN output in Fig. 5(b), which saves $f - n$ multipliers.

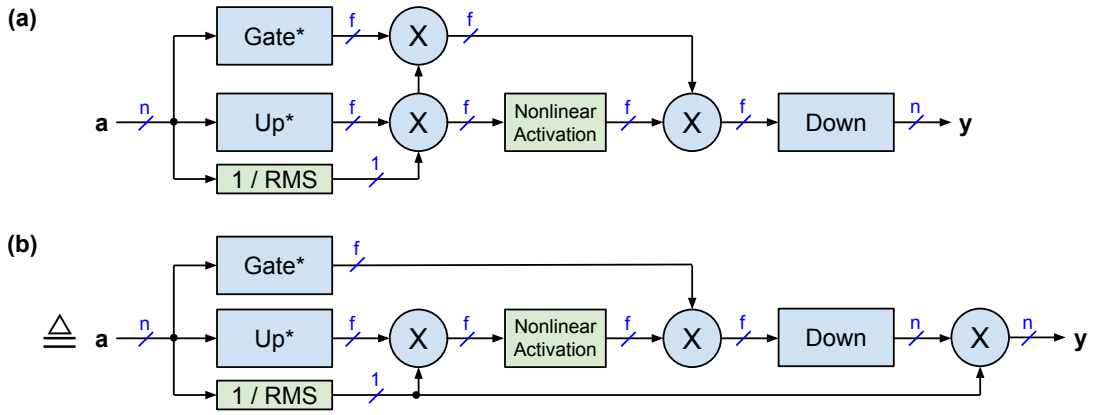

Figure 5: FFN with GLU variant and preceding flash normalization: (a) unoptimized version; (b) optimized version with fewer scaling multipliers. Gate, Up, and Down denote the linear layers for gate, up, and down projections.

Special case for ReGLU and Bilinear GLU: If the activation function is ReLU (aka ReGLU [17]) or just linear (aka bilinear GLU [17]), then we can also eliminate the scaling before the activation function and combine it with the scaling at the output as illustrated in Fig. 6(b), which saves $2f - n$ multipliers. Now the output scaling is using the reciprocal of the squared RMS as scaling value, which is the same as the reciprocal of the mean-square (MS):

$$\frac{1}{(\mathrm{RMS}(\vec{a}))^2} = \frac{1}{\mathrm{MS}(\vec{a})} = \frac{1}{\frac{1}{n}\sum_{i=1}^{n} a_i^2} = \frac{n}{\sum_{i=1}^{n} a_i^2}$$

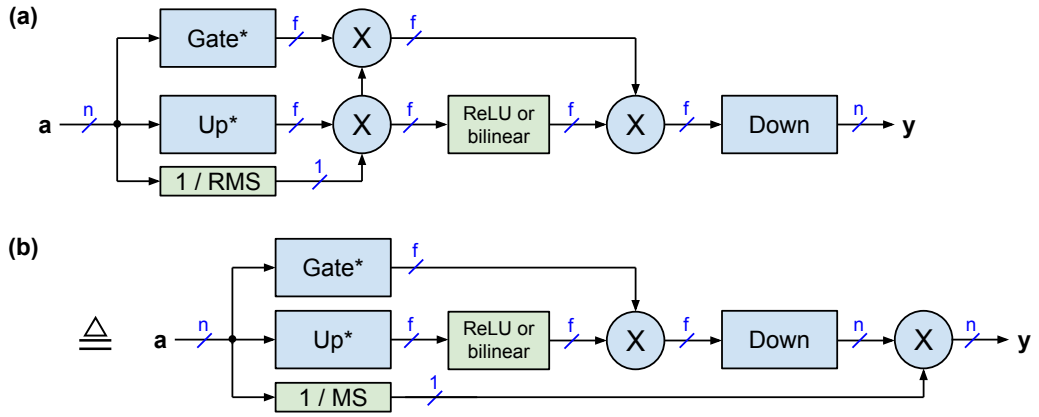

Figure 6: FFN with ReGLU (or bilinear GLU) and preceding flash normalization: (a) unoptimized version; (b) optimized version with fewer scaling multipliers.

3 Flash normalization for attention with RoPE

Fig. 7(a) shows the Q and K linear layers with flash normalization followed by RoPE [18] and scaled dot-product attention [13]. More details on Figure 7:

- Q* and K* are the linear layers for Q (queries) and K (keys) fused with the normalization weights of the activation vector $\vec{a}$ (according to flash normalization).
- h is the dimension of the attention heads.
- The boxes labeled cos, sin, and RoPE perform $\vec{y} = \vec{x} \cdot \cos(\cdot) + \text{permute}(\vec{x}) \cdot \sin(\cdot)$, where
  - $\text{permute}(\vec{x}) = (-x_2, x_1, -x_4, x_3, \ldots, -x_h, x_{h-1})$, see equation (34) of [18] for more details.
  - $\cos(\cdot) = (\cos m\theta_1, \cos m\theta_1, \cos m\theta_2, \cos m\theta_2, \ldots, \cos m\theta_{h/2}, \cos m\theta_{h/2})$ for position m.
  - $\sin(\cdot) = (\sin m\theta_1, \sin m\theta_1, \sin m\theta_2, \sin m\theta_2, \ldots, \sin m\theta_{h/2}, \sin m\theta_{h/2})$ for position m.
- Note that $\cos(\cdot)$ and $\sin(\cdot)$ only depend on the position of activation vector $\vec{a}$ and are shared among all attention heads. Therefore, it's more efficient to first scale $\cos(\cdot)$ and $\sin(\cdot)$ by $1/\text{RMS}(\vec{a})$ as illustrated in Fig. 7(b). This saves $2hH - h$ multipliers, where H is the number of attention heads.
- Furthermore, we can fuse the scaling factor $1/\sqrt{h}$ of the scaled dot-product with the $1/\text{RMS}(\vec{a})$ factor (note that we need to use $\sqrt{1/\sqrt{h}}$ as a scaling factor for this).
- Unfortunately, the V linear layer (value projection) still needs the normalization at its output.

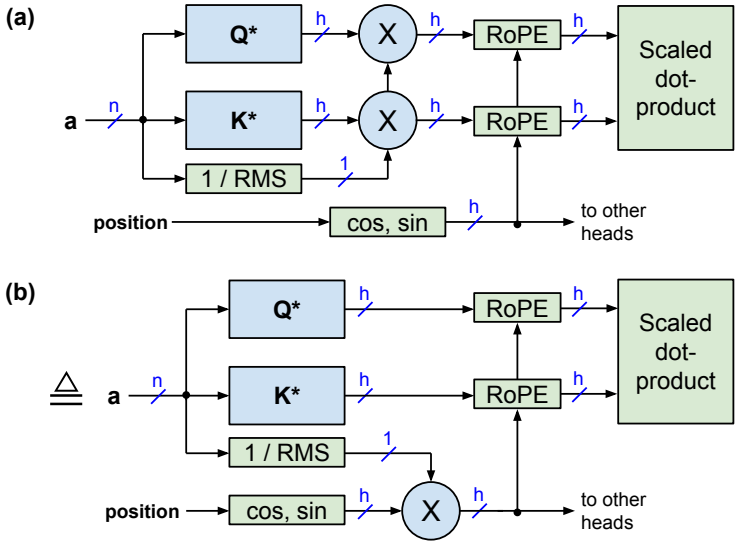

Figure 7: Flash normalization for scaled dot-product attention with RoPE: (a) unoptimized version; (b) optimized version where the normalization is fused with $\cos(\cdot)$ and $\sin(\cdot)$.

4 Optimizations for QK-normalization with RoPE

Gemma 3 [19], OLMo 2 [5], OpenELM [15] and other LLMs use query-key normalization (QK-norm) [20]. For example, each layer of Gemma 3 and OpenELM has the following two sets of normalization weights:

- `q_norm_weight`: query normalization weights for all heads of this layer
- `k_norm_weight`: key normalization weights for all heads of this layer

Unfortunately, FlashNorm can't be applied to QK-norm. But for the type of QK-norm used in Gemma 3 and OpenELM, we can apply the following two optimizations detailed in the next sections:

1. Eliminate the RMS calculation before the Q and K linear layers.
2. Fuse the normalization weights with RoPE.

4.1 Eliminate RMS calculation before QK linear layers

Fig. 8(a) shows a linear layer with flash normalization followed by an additional normalization. The weights of the first normalization are already merged into the linear layer weights $\mathbf{W}^*$. Note that $\text{RMS}(s \cdot \vec{a}) = s \cdot \text{RMS}(\vec{a})$ where s is scalar and $\vec{a}$ is a vector. Due to this scale-invariance of the RMS function, the second multiplier (scaler s_c) in the pipeline of Fig. 8(a) cancels out the first multiplier (scaler s_a). Fig. 8(b) takes advantage of this property. We can express this by using the vectors $\vec{a}, \vec{b}, \vec{c}$ along the datapath in Fig. 8 as follows:

- Note that $s_c = \frac{1}{\text{RMS}(\vec{c})} = \frac{1}{\text{RMS}(\vec{b} \cdot s_a)} = \frac{1}{s_a \cdot \text{RMS}(\vec{b})} = \frac{s_b}{s_a}$.
- With above, we can show that the y outputs of figures 8(a) and 8(b) are identical:

$$y = \vec{a} \cdot \mathbf{W}^* \cdot s_a \cdot s_c \cdot \vec{g} = \vec{a} \cdot \mathbf{W}^* \cdot s_a \cdot \frac{s_b}{s_a} \cdot \vec{g} = \vec{a} \cdot \mathbf{W}^* \cdot s_b \cdot \vec{g}$$

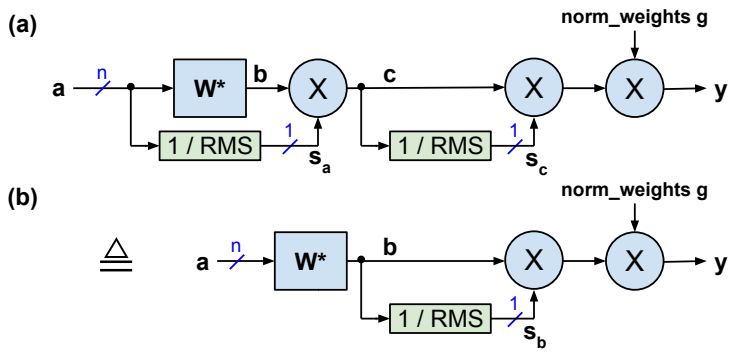

Figure 8: Linear layer with flash normalization followed by a second normalization: (a) unoptimized version; (b) optimized version.

The scale-invariance property of $\text{RMS}(\vec{a})$ doesn't hold exactly true for RMS with epsilon (see appendix). This should not matter because the epsilon only makes an impact if the RMS (or energy) of the activation vector is very small, in which case the epsilon limits the up-scaling of this low-energy activation vector.

4.2 Fuse normalization weights with RoPE

Fig. 9(a) illustrates QK-norm with RoPE. If the QK-norm weights are the same for all heads of a layer, as is the case for Gemma 3 and OpenELM [19, 15], then we can fuse them with RoPE's $\cos(\cdot)$ and $\sin(\cdot)$ as follows: multiply $\cos(\cdot)$ and $\sin(\cdot)$ with the normalization weights and then share the fused $\cos(\cdot)$ and $\sin(\cdot)$ vectors across all heads of the LLM layer as shown in Fig. 9(b). This requires permutation of the normalization weights $\vec{g}$ so that the boxes labeled cos, sin, and RoPE in Fig. 9(b) perform $\vec{y} = \vec{x} \cdot (\cos(\cdot) \cdot \vec{g}) + \text{permute}(\vec{x}) \cdot (\sin(\cdot) \cdot \text{permuteg}(\vec{g}))$, where $\text{permuteg}(\vec{g}) = (g_2, g_1, g_4, g_3, \ldots, g_h, g_{h-1})$. For simplicity, Fig. 9(b) doesn't show the permutation of the normalization weights.

5 Bottleneck of RMS normalization for batch 1

This section describes the compute bottleneck of RMS normalization that exists for batch size 1. This bottleneck is different from the bottleneck detailed in [15]. Let's consider a processor with one vector unit and one matrix unit:

- The matrix multiplications of the linear layers are performed by the matrix unit, while the vector unit performs vector-wise operations such as RMSNorm and FlashNorm.
- Let's assume that the vector unit can perform m operations per cycle and the matrix unit can perform m^2 operations per cycle, where m is the processor width. Specifically:
  - Multiplying an n-element vector with an $n \times n$ matrix takes n^2 MAD (multiply-add) operations, which takes n^2/m^2 cycles with our matrix unit.

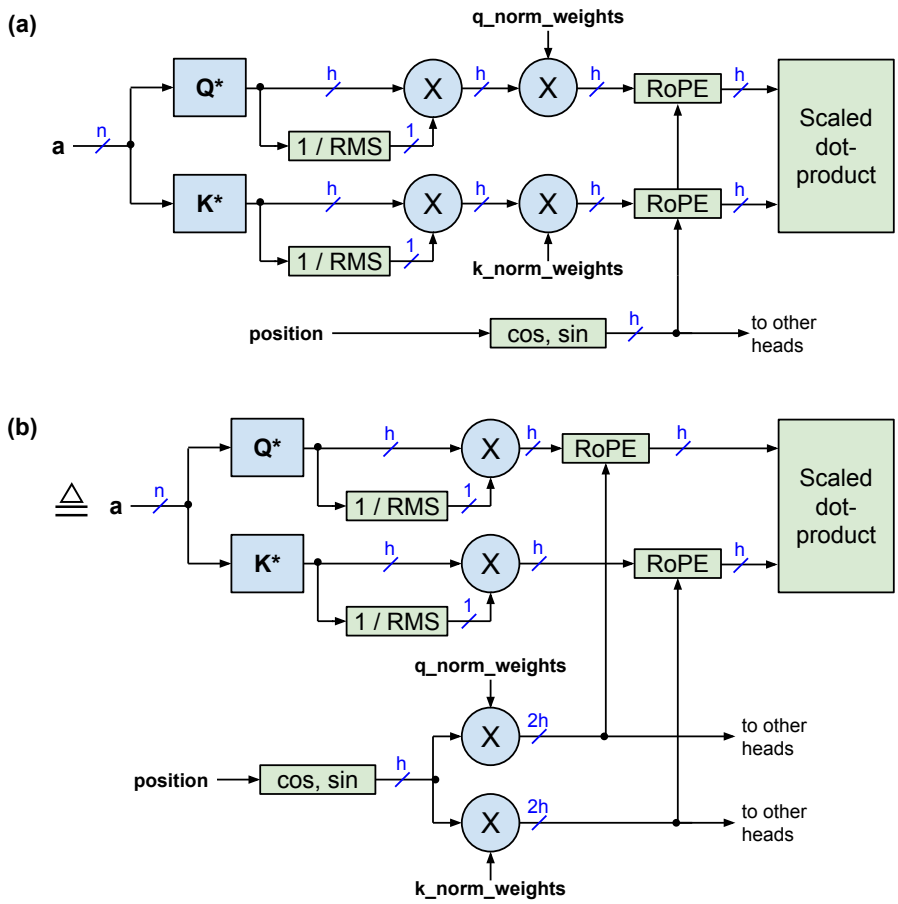

Figure 9: QK-norm with RoPE: (a) unoptimized version; (b) optimized version.

- Calculating $1/\text{RMS}(\vec{a})$ takes n MAD operations (for squaring and adding) plus 2 scalar operations (for $\sqrt{n/x}$), which takes n/m cycles with our vector unit if we ignore the 2 scalar operations.

- Scaling an n-element vector by a scaling factor takes n multiply operations, which takes n/m cycles.

For the example $n = 512, m = 128$ and batch 1, Fig. 10 shows timing diagrams without and with deferred normalization:

- Without deferred normalization, the matrix unit has to wait for 8 cycles until the vector unit has calculated the RMS value and completed the scaling by $1/\text{RMS}(\vec{a})$ as illustrated in Fig. 10(a).

- As shown in Fig. 10(b), it is possible to start the matrix unit 3 cycles earlier if the weight matrix $\mathbf{W}$ is processed in row-major order for example. But the RMS calculation still presents a bottleneck.

- FlashNorm eliminates this bottleneck: With deferred normalization, the matrix unit computes the vector-matrix multiplication in parallel to the vector unit's RMS calculation as shown in Fig. 10(c). The scaling at the end can be performed in parallel to the matrix unit if $\mathbf{W}$ is processed in column-major order for example.

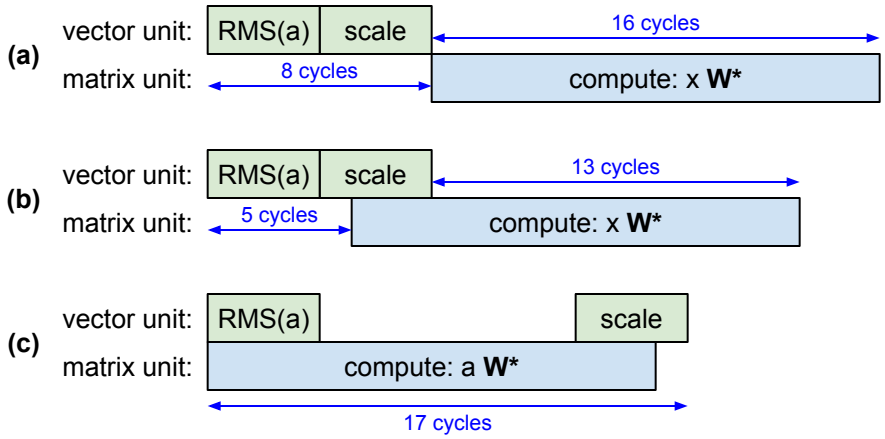

Figure 10: Timing diagrams for $n = 512, m = 128$: (a) without deferred normalization; (b) with interleaved scaling and vector-matrix multiplication; (c) with deferred normalization.

6 Experiments and conclusions

Refer to [21, 10] for Python code that demonstrates the mathematical equivalency of the optimizations presented in this paper. A reproducible Colab notebook for the GPU benchmarks below is available in the `transformer-tricks` repository[2].

We verified lossless weight folding on SmolLM2-135M [22] using `flashify_repo()` from the `transformer-tricks` package [10]. The original and weight-folded models produce identical logits (max difference = 0.0, cosine similarity = 1.0) and identical generated text.

To validate the parallel execution described in Fig. 10, we benchmarked the norm-then-project operation on an NVIDIA T4 GPU (65 TFLOPS FP16, 320 GB/s). We compare sequential execution (RMSNorm $\rightarrow$ GEMM) against FlashNorm (GEMM $\parallel$ $1/\text{RMS}(\vec{a}) \rightarrow$ scale), where the GEMM runs on tensor cores and $1/\text{RMS}(\vec{a})$ runs on vector cores via a Triton kernel on separate CUDA streams. Table 1 shows the results at SmolLM2-135M scale, and Table 2 at Llama-7B scale.

Table 1: Norm+project latency on T4 GPU at SmolLM2-135M scale (n=576, k=960).

Tokens	Sequential	FlashNorm	Speedup	Regime
1	0.374 ms	0.616 ms	−64.8%	decode (mem-bound)
64	0.506 ms	0.737 ms	−45.8%	decode (mem-bound)
4096	0.704 ms	0.468 ms	+33.6%	prefill (compute-bound)
8192	0.929 ms	0.599 ms	+35.5%	prefill (compute-bound)

Table 2: Norm+project latency on T4 GPU at Llama-7B scale (n=4096, k=4096).

Tokens	Sequential	FlashNorm	Speedup
1	0.641 ms	0.927 ms	−44.6%
64	0.307 ms	0.644 ms	−109.8%
256	0.577 ms	0.546 ms	+5.5%
1024	1.882 ms	1.654 ms	+12.1%
4096	7.628 ms	6.570 ms	+13.9%

The speedup appears only when the GEMM is compute-bound (many tokens, i.e. prefill). For memory-bound GEMMs (few tokens, i.e. decode), stream management overhead dominates. Profiling individual operations within a SmolLM2-135M decoder layer at 4096 tokens confirms that the $1/\text{RMS}(\vec{a})$ calculation takes $\sim$0.08 ms per layer. Across 30 layers, the theoretical maximum savings from perfect overlap is $\sim$5 ms out of $\sim$60 ms total ($\sim$8%).

[2] https://github.com/OpenMachine-ai/transformer-tricks/blob/main/notebooks/flashNorm_gpu_benchmark.ipynb

While the speedup on the norm+project operation itself is significant (33–35%), the end-to-end model speedup is modest because this operation is a fraction of total layer time. For comparison, we measured a throughput of 204 tokens per second for OpenELM-270M with 4-bit weight quantization using the MLX framework on an M1 MacBook Air. This throughput increases to only 225 tokens per second when we remove RMSNorm entirely, giving an upper bound of $\leq 10\%$ for this model. Native integration into inference frameworks at the kernel level—where stream management overhead is negligible—would allow the 33–35% operation-level improvement to translate more directly into end-to-end gains, particularly for larger models with bigger GEMMs.

For many applications, the main advantage of FlashNorm is simplification. This is similar to the simplifications we get from using RMSNorm over Layer Normalization (LayerNorm [6]), and from PaLM's removal of bias-parameters from all linear layers [23].

Future work includes integrating FlashNorm into popular frameworks such as HuggingFace Transformers [24], whisper.cpp [25], llama.cpp [26], vLLM [27], llamafile [28], LM Studio [29], Ollama [30], SGLang [31], and combining it with parameter quantization.

Acknowledgments

We would like to thank Dmitry Belenko for helpful feedback on this work.

A RMS with epsilon

Many implementations add a small epsilon ϵ to the RMS value to limit the resulting scaling factor $1/\text{RMS}(\vec{a})$ and to avoid division by zero as follows:

$$\text{RMSe}(\vec{a}) = \sqrt{\epsilon + \frac{1}{n}\sum_{i=1}^{n} a_i^2} = \sqrt{\epsilon + (\text{RMS}(\vec{a}))^2}$$

$\text{RMSe}(\vec{a})$ can be used as a drop-in-replacement for RMS. The popular HuggingFace transformer library calls this epsilon `rms_norm_eps`, which is set to 10^{-5} for Llama3.

B Eliminating $1/n$

This section details a small optimization that eliminates the constant term $1/n$ from the RMS calculation. First, we factor out $1/n$ as follows:

$$\text{RMS}(\vec{a}) = \sqrt{\frac{1}{n}\sum_{i=1}^{n} a_i^2} = \sqrt{\frac{1}{n}}\sqrt{\sum_{i=1}^{n} a_i^2} = \sqrt{\frac{1}{n}} \cdot \text{RSS}(\vec{a})$$

where $\text{RSS}(\vec{a}) = \sqrt{\sum_{i=1}^{n} a_i^2}$. We can now merge the constant term into the normalization weights g_i as follows:

$$y_i = \frac{a_i}{\text{RMS}(\vec{a})} \cdot g_i = \frac{a_i}{\text{RSS}(\vec{a})}\sqrt{n} \cdot g_i = \frac{a_i}{\text{RSS}(\vec{a})} \cdot g_i^*$$

with new normalization weights $g_i^* = \sqrt{n} \cdot g_i$. These new normalization weights can now be merged with the weights $\mathbf{W}$ of the following linear layer as shown in the previous sections. This optimization also applies for the case where we add an epsilon as detailed in the previous section. In this case, we factor out $1/n$ as follows:

$$\text{RMSe}(\vec{a}) = \sqrt{\epsilon + \frac{1}{n}\sum_{i=1}^{n} a_i^2} = \sqrt{\frac{1}{n}\left(n\epsilon + \sum_{i=1}^{n} a_i^2\right)} = \sqrt{\frac{1}{n}} \cdot \text{RSSe}(\vec{a})$$

where $\text{RSSe}(\vec{a}) = \sqrt{n\epsilon + \sum_{i=1}^{n} a_i^2}$.

References

[1] Biao Zhang and Rico Sennrich. Root mean square layer normalization. October 2019. *arXiv:1910.07467*.

[2] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. LLaMA: Open and efficient foundation language models. February 2023. *arXiv:2302.13971*.

[3] Gemma Team, Google DeepMind. Gemma: Open Models Based on Gemini Research and Technology. 2024.

[4] Albert Q Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Lélio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. Mistral 7B. October 2023. *arXiv:2310.06825*.

[5] Team OLMo, Pete Walsh, Luca Soldaini, Dirk Groeneveld, Kyle Lo, Shane Arora, Akshita Bhagia, Yuling Gu, Shengyi Huang, Matt Jordan, Nathan Lambert, Dustin Schwenk, Oyvind Tafjord, Taira Anderson, David Atkinson, Faeze Brahman, Christopher Clark, Pradeep Dasigi, Nouha Dziri, Michal Guerquin, Hamish Ivison, Pang Wei Koh, Jiacheng Liu, Saumya Malik, William Merrill, Lester James V Miranda, Jacob Morrison, Tyler Murray, Crystal Nam, Valentina Pyatkin, Aman Rangapur, Michael Schmitz, Sam Skjonsberg, David Wadden, Christopher Wilhelm, Michael Wilson, Luke Zettlemoyer, Ali Farhadi, Noah A Smith, and Hannaneh Hajishirzi. 2 OLMo 2 Furious. December 2024. *arXiv:2501.00656*.

[6] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer Normalization. July 2016. *arXiv:1607.06450*.

[7] Jiachen Zhu, Xinlei Chen, Kaiming He, Yann LeCun, and Zhuang Liu. Transformers without Normalization. 2025. *arXiv:2503.10622*.

[8] Christopher Marquand with audio generated by Notebook LM. Podcast-video about FlashNorm. May 2025.

[9] Nils Graef and Andrew Wasielewski. Slim attention: cut your context memory in half without loss – K-cache is all you need for MHA. March 2025. *arXiv:2503.05840*.

[10] OpenMachine. Transformer tricks. 2024. URL `https://github.com/OpenMachine-ai/transformer-tricks`.

[11] Nils Graef. KV-weights are all you need for skipless transformers. April 2024. *arXiv:2404.12362*.

[12] Nils Graef. Transformer tricks: Precomputing the first layer. February 2024. *arXiv:2402.13388*.

[13] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. June 2017. *arXiv:1706.03762*.

[14] Tri Dao, Daniel Y Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. May 2022. *arXiv:2205.14135*.

[15] Sachin Mehta, Mohammad Hossein Sekhavat, Qingqing Cao, Maxwell Horton, Yanzi Jin, Chenfan Sun, Iman Mirzadeh, Mahyar Najibi, Dmitry Belenko, Peter Zatloukal, and Mohammad Rastegari. OpenELM: An efficient language model family with open-source training and inference framework. April 2024. *arXiv:2404.14619*.

[16] Wikipedia. Rectifier (neural networks), 2024. Accessed June-2024.

[17] Noam Shazeer. GLU Variants Improve Transformer. February 2020. *arXiv:2002.05202*.

[18] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced transformer with Rotary Position Embedding. April 2021. *arXiv:2104.09864*.

[19] Gemma Team, Aishwarya Kamath, Johan Ferret, Shreya Pathak, Nino Vieillard, Ramona Merhej, Sarah Perrin, Tatiana Matejovicova, Alexandre Ramé, Morgane Rivière, Louis Rouillard, Thomas Mesnard, Geoffrey Cideron, Jean-Bastien Grill, Sabela Ramos, Edouard Yvinec, Michelle Casbon, Etienne Pot, Ivo Penchev, Gaël Liu, Francesco Visin, Kathleen Kenealy, Lucas Beyer, Xiaohai Zhai, Anton Tsitsulin, Robert Busa-Fekete, Alex Feng, Noveen Sachdeva,

Benjamin Coleman, Yi Gao, Basil Mustafa, Iain Barr, Emilio Parisotto, David Tian, Matan Eyal, Colin Cherry, Jan-Thorsten Peter, Danila Sinopalnikov, et al. Gemma 3 Technical Report. March 2025. *arXiv:2503.19786*.

[20] Alex Henry, Prudhvi Raj Dachapally, Shubham Pawar, and Yuxuan Chen. Query-key normalization for transformers. October 2020. *arXiv:2010.04245*.

[21] OpenMachine. FlashNorm. 2024. URL https://huggingface.co/open-machine/FlashNorm.

[22] Loubna Ben Allal, Anton Lozhkov, Elie Bakouch, Gabriel Martín Blázquez, Lewis Tunstall, Agustín Piqueres, Andres Marafioti, Cyril Zakka, Leandro von Werra, and Thomas Wolf. SmolLM2 - with great data, comes great performance, 2024. URL https://huggingface.co/HuggingFaceTB/SmolLM2-1.7B.

[23] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, et al. PaLM: Scaling language modeling with Pathways. April 2022. *arXiv:2204.02311*.

[24] HuggingFace. Transformers. URL https://huggingface.co/docs/transformers.

[25] Georgi Gerganov. whisper.cpp. . URL https://github.com/ggml-org/whisper.cpp.

[26] Georgi Gerganov. llama.cpp. . URL https://github.com/ggml-org/llama.cpp.

[27] vLLM Project. vLLM. URL https://github.com/vllm-project/vllm.

[28] Mozilla. llamafile. URL https://github.com/Mozilla-Ocho/llamafile.

[29] LM Studio. LM Studio. URL https://lmstudio.ai.

[30] Ollama. Ollama. URL https://github.com/ollama/ollama.

[31] SGLang. SGLang. URL https://github.com/sgl-project/sglang.